

Selected Topics in Management Science

Dr. Mustafa Çimen

Lecture Notes

MAN403 - Selected Topics in Management Science

Fall 2026



Contents

- 1 Introduction
 - The Course
 - Why Heuristics?
 - Policies, Not Plans
 - Testing a Rule by Simulation
 - The Eight Companies



What This Course Is About

- In MAN305 you built mathematical models and let Gurobi find the optimum.
- Real problems break this in two ways:
 - they are **too large**: the model does not solve in any reasonable time;
 - they are **uncertain**: some data is only known once it happens.
- The practical answer is a good **heuristic**: a method that finds good, not necessarily optimal, decisions fast.
- This course studies three families of heuristics, one per block:
 - ① heuristics built on LP/MIP,
 - ② heuristics built on dynamic programming,
 - ③ heuristics built on simulation and learning.



Three Blocks

Weeks	Block	Project due
1	Introduction; the eight problems; evaluating a policy	
2–5	LP/MIP-based heuristics: rolling horizon, fix-and-optimize, decomposing a big decision into smaller ones, dropping hard constraints and repairing, matheuristics	Week 6
6–9	DP-based heuristics: restricted DP, simulation-based restricted DP	Week 10
10–13	Simulation- and learning-based heuristics: TD learning, SARSA, Q-learning, function approximation	Week 14
14	Project Market	



Assessment

Project 1 (LP/MIP-based heuristic)	15%
Project 2 (DP-based heuristic)	15%
Project 3 (TD-learning heuristic)	15%
Attendance	5%
Final: comparative report + oral exam	50%

- **One problem per student**, kept for the whole semester.
- Each project solves *your* problem with the technique of that block: Python code + a report written as an academic article.
- **Final:** compare your three heuristics, using your classmates' findings as well, and answer my questions on them in an oral exam.



Project Market

- In Week 14 (22.12.2026) we hold a **Project Market**.
- Each of you presents a **poster**: your problem, your three heuristics, and how they compare.
- The audience: department students and alumni.
- A good poster tells a story in one look:
 - What is the decision?
 - What makes it hard?
 - Which method won, by how much, and at what computing cost?



Tools and Rules

- **Python** for everything; **Gurobi** (gurobipy) for Project 1; Visual Studio Code as the editor.
- You are assumed to know LP, MIP and dynamic programming (MAN305 and your previous elective).
- **Using AI tools is allowed**, on one condition: **you direct the tool, it does not direct you**. In every project I will ask you to change your own method in front of me — add a constraint, change a cost, make the rule behave differently in one situation.
- So explain your work **in your own words**, and **cite everything you use**: code, figures, text and ideas.



Why Exact Methods Fail (1): Size

- Take Glorfindel from the project booklet: one tanker, 15 petrol stations, 10 days.
- As a dynamic program, the **state** is the stock level of every station.
 - Say each tank can hold 0 to 38 units: 39 possible stock levels per station.
 - 15 stations: $39^{15} \approx 7 \times 10^{23}$ states.
- The **decision**: which stations to visit ($2^{15} = 32\,768$ subsets), how much to deliver to each, and the route.
- The **outcome**: the joint demand of 15 stations.
- No computer enumerates this. Powell calls these the **three curses of dimensionality**: state, decision and outcome [2].



Why Exact Methods Fail (2): Uncertainty

- A MIP needs numbers. When demand is uncertain, the tempting shortcut is to plug in the **average**.
- The plan is then optimal for a future that never happens exactly.
- Costs are usually **asymmetric and non-linear** in the uncertain data: too much stock costs a little, too little stock costs a lot.
- Then the cost of the average scenario is *not* the average cost. This is the **flaw of averages** [4].



The Flaw of Averages: A One-Station Example

- Daily demand $D \sim \text{Poisson}(20)$. Holding: 1 per leftover unit; lost sale: 10 per unit.
- **Plan with the average:** stock $S = 20$ units. The plan predicts a cost of 0.
- **What actually happens**, on average:

$$\begin{aligned} E[\text{cost}] &= 1 \cdot E[(20 - D)^+] + 10 \cdot E[(D - 20)^+] \\ &= 1.78 + 17.77 = \mathbf{19.54} \end{aligned}$$

- **Best stock level** under uncertainty: $S = 26$, expected cost **8.41**.
- Planning with the average cost more than twice as much as it had to.



In-class Exercise 1: Choosing a Stock Level

A shop's daily sales are 8, 12 or 18 units, with probabilities 0.25, 0.50 and 0.25. Leftover stock costs 1 per unit; a lost sale costs 10 per unit. The shop starts the day with S units.

Write a loop that computes the expected cost of a stock level S , and run it for $S = 12, 15$ and 18. Then run it for every S from 8 to 18 — is the best one in the table?

S	cost if sales turn out to be			expected cost
	8 ($p=0.25$)	12 ($p=0.50$)	18 ($p=0.25$)	
12	4 left: 4	nothing left: 0	6 lost: 60	?
15	7 left: 7	3 left: 3	3 lost: 30	?
18	10 left: 10	6 left: 6	nothing lost: 0	?



In-class Exercise 1: Answers

- $S = 12$: $0.25(4) + 0.50(0) + 0.25(60) = \mathbf{16.0}$
- $S = 15$: $0.25(7) + 0.50(3) + 0.25(30) = \mathbf{10.75}$
- $S = 18$: $0.25(10) + 0.50(6) + 0.25(0) = \mathbf{5.5}$ (best of the three)

Run the loop over every S from 8 to 18 and **18 is still the winner** — worth knowing, because three hand-picked values could easily have missed it.

- The best stock level is *above* the average demand of 12.5, because a lost sale hurts ten times more than a leftover unit.
- No average could have told you this: the whole **table of possibilities** matters.
- And this is one shop on one day. Your company has many locations and many days.

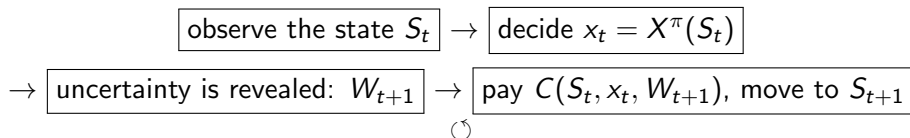


Plan vs Policy

- A **plan** fixes every decision in advance: “deliver 30 units to station 4 on Tuesday”.
- A **policy** is a rule that makes the next decision from what is known *at that moment*: “on any morning, if station 4 has less than one day of stock, fill it up”.
- In a multi-stage uncertain problem, information arrives over time. A policy uses it; a plan ignores it.
- In this course **every heuristic you build is a policy**.



The Sequential Decision Cycle



- X^π is the decision rule of the policy π .
- Goal: find the policy with the smallest **expected** total cost

$$\min_{\pi} E \left[\sum_t C(S_t, X^\pi(S_t), W_{t+1}) \right].$$

- This is the Markov decision process you know from dynamic programming; the notation follows [2].



Three Ways to Build a Policy

Block 1 Optimise a model at every step. Solve a (deterministic, expected-value) MIP over the next few periods, apply the first decision, roll forward. Make the MIP tractable with fix-and-optimize, decomposition or relaxation.

Block 2 Search the stages, but not all states. Dynamic programming stage by stage, keeping only the most promising states (restricted DP). Under uncertainty, judge a state by simulated scenarios.

Block 3 Learn from simulated experience. Let the policy play the problem thousands of times in a simulator and learn the value of states and actions with temporal-difference methods.



How Good Is a Decision Rule?

- Because the future is uncertain, the same rule gives a different cost on every run.
- So we ask: what does the rule cost **on average**?
- We find out by **simulation**:
 - 1 play the company's problem day by day on the computer,
 - 2 let the rule make every decision,
 - 3 draw the uncertain values (sales, travel times, ...) at random,
 - 4 add up the costs;
 - 5 repeat for 100 runs and take the average.



A Simulator in Plain Python

```
import random

def simulate(rule, days=10):
    stock = 30
    total_cost = 0
    for day in range(days):
        order = rule(stock)                # the decision rule decides
        stock = stock + order
        sales = random.choices([8, 12, 18], weights=[0.25, 0.50, 0.25])[0]
        if sales > stock:
            total_cost = total_cost + 10 * (sales - stock)    # lost sales
            stock = 0
        else:
            stock = stock - sales
        total_cost = total_cost + 1 * stock    # cost of keeping stock
        if order > 0:
            total_cost = total_cost + 20      # cost of a delivery
    return total_cost
```



Testing a Rule on 100 Test Cases

```
def simple_rule(stock):  
    if stock < 12:                # less than one normal day left  
        return 36 - stock        # fill up to three normal days  
    return 0  
  
costs = []  
for k in range(1, 101):  
    random.seed(k)                # run k: always the same "days"  
    costs.append(simulate(simple_rule))  
  
print("average cost:", sum(costs) / len(costs))  
print("highest cost:", max(costs))
```

- `random.seed(k)` makes run k the same every time you run it.
- To test a new rule, write another function like `simple_rule` and run the same loop.



Comparing Two Rules Fairly

- Test both rules on the **same** 100 runs (the same seeds).
- Then both rules face the same busy days and the same quiet days. A difference in cost comes from the rules, not from luck.
- Compare them case by case as well: in how many of the 100 cases is the new rule cheaper?
- How sure are we about an average? Your statistics course gives the answer: the 95% confidence interval $\bar{c} \pm 1.96 s/\sqrt{100}$ [1].



Rules for Honest Tests

- **Test cases:** seeds $1, \dots, 100$, the same for every method.
- **Trying out and tuning** a method: other seeds, for example 1000 and above. If you tune on those runs, your results will look better than they really are.
- Always show the **simple starting rule** next to your method.
- Always report the **computing time**.



What the Simulator Actually Did: Test Case 1

Starting stock 30; the rule “if stock < 12 , fill up to 36”; sales drawn from 8/12/18.

Day	Delivered	Sales	Lost sales	Stock at night	Cost so far
1	0	8	0	22	22
2	0	18	0	4	26
3	32	18	0	18	64
4	0	12	0	6	70
5	30	12	0	24	114
6	0	12	0	12	126
7	0	12	0	0	126
8	36	18	0	18	164
9	0	8	0	10	174
10	26	8	0	28	222

This is one run out of 100. Notice day 7: the shop ends the day with nothing left, and is lucky.



Comparing Two Rules: Real Numbers

Both rules tested on the same 100 runs:

Rule	Average cost	Highest cost	Improvement
Fill up to 36 when stock is below 12	227.3	298	–
Fill up to 30 when stock is below 12	212.7	306	6.4%

- The second rule is 6.4% cheaper **on average**...
- ...but it is cheaper in only **52 of the 100** runs, and its **worst** case is worse.
- So always report more than the average. A rule that is a little cheaper on average but much worse on a bad day may not be the one the company wants.



Eight Companies, One per Student

	Company and decision	What is not known in advance
P1	Glorfindel : supplying 15 petrol stations with one tanker	daily fuel sales
P2	Fossoglor : what to produce on one line, 8 products	weekly orders
P3	Milkboy : fresh milk to 12 supermarkets, 2 trucks	daily milk sales (milk expires in 3 days)
P4	Oronar : orders in a four-stage supply chain	weekly customer demand
P5	Findel : collecting empty crates from 30 shops	crates at each shop
P6	Ronar : 3 technicians, 25 promised visit windows	travel times
P7	Glor : the print shop's next job	which orders arrive, and when
P8	Fossoronar : 40 jobs on 4 machines	how long each job takes

Full stories: **Project Booklet**.



What Every Company Has in Common

- The same kind of decision is made **again and again**: every day, every week, or every time a machine becomes free.
- Something important is **not known in advance**.
- **You choose the numbers** (demands, costs, distances, . . .) and explain where they come from. The booklet only fixes the size.
- Each story ends with a **simple starting rule**. Program it first; every method you build is judged by how much it improves on it.
- Each company can be helped with all three blocks' methods, so by Week 14 you have three answers to the same question.



Choosing Your Numbers: A Worked Example (Glorfindel)

Stations	15 points on a 20×20 km map, depot in the middle (drawn on squared paper)
Average daily sales	5 to 20 thousand litres, depending on the size of the station
Uncertainty	quiet $0.7\times$, normal $1\times$, busy $1.4\times$ the average ($0.25 / 0.50 / 0.25$)
Tank size	three days of average sales
Tanker capacity	1.5 days of the sales of all stations together
Cost per km	25 MU (fuel and driver)
Holding cost	0.02 MU per litre per day (money tied up in stock)
Lost sale	1.5 MU per litre (lost margin and an annoyed customer)

Nothing here is “the right answer”. What matters is that the numbers are sensible and that you can explain where they come from.



For Next Week

- 1 Read your company's story in the Project Booklet.
- 2 Choose your numbers, and write down where each one comes from.
- 3 Describe what is uncertain with a small table of values and probabilities.
- 4 Write your **simulator** and the **simple starting rule**, and test the rule on the 100 runs.

You will use these pieces in all three projects, so it pays to build them carefully once.



References



Law, A. M. (2015). *Simulation Modeling and Analysis* (5th ed.). New York: McGraw-Hill.



Powell, W. B. (2011). *Approximate Dynamic Programming: Solving the Curses of Dimensionality* (2nd ed.). Hoboken, NJ: Wiley.



Ropke, S. & Pisinger, D. (2006). An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows. *Transportation Science*, 40(4), 455–472.



Savage, S. L. (2009). *The Flaw of Averages: Why We Underestimate Risk in the Face of Uncertainty*. Hoboken, NJ: Wiley.



Sutton, R. S. & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). Cambridge, MA: MIT Press.

Worked examples without a reference (the one-station example, the Glorfindel state-space count) were computed for these slides.



Thanks for listening!

See you in the next lecture...

Dr. Mustafa Çimen

